

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile
To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

```
varshini@VarshiniBolla:~$ ./myshell
myshell> nano s1.c
myshell> gcc s1.c -o shell
myshell> ./shell
Linux OSSP project is ready.
GCC and Makefile can be used to build this program.
```

2. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```
myshell> nano s2.c
myshell> gcc s2.c -o sl
myshell> ./sl
myshell>
You entered:
myshell> exit
Shell exited
```

3. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
myshell> nano s3.c
myshell> gcc s3.c -o sl
myshell> ./sl
Enter commands (type history to display, exit to stop).
> history
> exit
```

4. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
myshell> nano s4.c
myshell> gcc s4.c -o sl
myshell> ./sl
Enter command: ls
Token: ls
```

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
myshell> gcc s5.c -o ab
myshell> ./ab
Single quoted string: 'hello $USER world'
Content inside single quotes is treated literally.
```

6. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
myshell> nano s6.c
myshell> gcc s6.c -o shell
myshell> ./shell
Parsed: hello world
```

-
7. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```
myshell> gcc s7.c -o sl
myshell> ./sl
Parent waiting using waitpid()...
Child finished.
Child has terminated.
```

8. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
myshell> nano s8.c
myshell> gcc s8.c -o sl
myshell> ./sl
Enter variable name: 12
Variable is undefined.
```

-
9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

```
myshell> nano s9.c
myshell> gcc s9.c -o sl
myshell> ./sl
Enter directory: os lab
chdir: No such file or directory
```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
myshell> nano s10.c
myshell> gcc s10.c -o sl
myshell> ./sl
Current directory: /home/varshini
Type 'exit' in a shell to process an exit request.
```

11. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
myshell> nano s11.c
myshell> nano s11.c
myshell> gcc s11.c -o sl
myshell> ./sl
Command 1: ls
Command 2: exit
Command 3: ^C
```

12. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
varshini@VarshiniBolla:~$ nano s13.c
varshini@VarshiniBolla:~$ gcc s13.c -o sl
varshini@VarshiniBolla:~$ ./sl
input.txt: No such file or directory
```

13. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
myshell> nano s14.c
myshell> gcc s14.c -o sl
myshell> ./sl
```

14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
myshell> nano s15.c
myshell> gcc s15.c -o sl
```

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
myshell> nano s16.c
myshell> gcc s16.c -o sl
myshell> ./sl
Background job started. PID=542
Parent returns to prompt immediately.

Background job completed.
myshell> myshell> |
```

-
17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```
myshell> myshell> nano s17.c
myshell> gcc s17.c -o sl
myshell> ./sl
[1] PID=596 Status=Running
[1] PID=596 Status=Completed
```

-
18. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
myshell> nano s18.c
myshell> gcc s18.c -o sl
myshell> ./sl
PID: 609
Send SIGUSR1 using: kill -USR1 PID
```

-
19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
myshell> nano s19.c
myshell> gcc s19.c -o sl
myshell> ./sl
PID: 647
```

20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
myshell> gcc s20.c -o shell
myshell> ./shell
PID: 674
Press Ctrl+Z to suspend.
Running...
Running...
Running...
Running...
Running...
^Z
SIGTSTP received. Process will stop.

[1]+  Stopped                  ./myshell
```

21. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
myshell> nano s21.c
myshell> gcc s21.c -o sl
myshell> ./sl
Module 1 result: 30
End-to-end module test completed.
```

22. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
myshell> gcc s22.c -o sl
myshell> ./sl
Memory allocated successfully.
Memory released successfully.
```

23. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
myshell> gcc s23.c -o sl
myshell> ./sl
Pipeline job 1, PID=721
Pipeline job 2, PID=722
Pipeline job 3, PID=723
All jobs completed.
CPU time: 0.001 seconds
```

24. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
myshell> gcc s24.c -o sl
myshell> ./sl
Project structure check
src/      - source files
include/  - header files
build/    - compiled files
README.md - project documentation
Makefile  - build instructions
```